UT2. PROGRAMACIÓN MULTIHILO

EXAMEN PRÁCTICO 09/11/2026

Descripción:

- Dispones hasta que termine la sesión para completar el examen.
- El examen consta de 2 programas, evaluados de acuerdo con la puntuación que se indica en cada uno de los ejercicios.

Ejercicio 1. Gestión de hilos en un simulador de tráfico (5 puntos):

Eres desarrollador de una herramienta para simular el tráfico en una ciudad. En esta simulación, hay vehículos que deben avanzar por diferentes rutas de manera concurrente. Tu tarea consiste en gestionar los hilos que representan estos vehículos y observar sus estados.

Instrucciones:

a) **(1 punto) Creación de hilos extendiendo la clase Thread (Moto.java)**:
   i. Crea la **clase Moto** que extienda de Thread y reciba su número (id) en el constructor.
   ii. Cada hilo de esta clase representará un vehículo de tipo moto.
   iii. El hilo simulará una moto que avanza por una ruta y mostrará en consola su progreso, por ejemplo: "Moto X avanzando al punto Y". Donde X es el identificador de la moto e Y el punto actual.
   iv. Habrá 10 puntos intermedios de progresión de la moto.
   v. Simula un tiempo intermedio de progresión con Thread.sleep(500).
   vi. Al finalizar los 10 puntos, muestra un mensaje: "Moto X ha completado su recorrido".
   vii. Incluye el manejo de excepciones.

b) **(1,5 puntos) Creación de hilos implementando la interfaz Runnable (Coche.java):**
   i. Crea la **clase Coche**, que implemente Runnable y reciba su número (id) en el constructor.
   ii. Cada hilo de esta clase representará un vehículo de tipo coche.
   iii. El hilo simulará un coche que avanza por una ruta y mostrará en consola su progreso, por ejemplo: "Coche X avanzando al punto Y"
   iv. Habrá 10 puntos intermedios de progresión del coche.
   v. Simula un tiempo intermedio de progresión con Thread.sleep(450).
   vi. Al finalizar los 10 puntos, muestra un mensaje: "Coche X ha completado su recorrido".
   vii. Incluye el manejo de excepciones.

c) **(1,5 puntos) Ejecución de hilos y visualización de información básica (SimuladorTrafico.java)**:

- Escribe el código necesario para iniciar **dos motos y dos coches** (moto1, moto2, coche1, coche2) y que cada uno muestre su progreso por consola.
- Recuerda que los coches deben ser de tipo Thread: Thread coche1 = new Thread(new Coche(1));
- Asegúrate de llamar al método start() para iniciar los hilos y verifica que cada hilo se ejecuta de forma independiente, comprobando en la consola que los mensajes de los cuatro vehículos aparecen intercalados.

d) **(1 punto) Identificación de los estados de los hilos (SimuladorTrafico.java):**

Con **getState()**, muestra el estado de los cuatro hilos en tres momentos:

1. Antes de llamar a start().
2. Durante la ejecución, con el bucle de la sugerencia.
3. Después del bucle.

Al final, muestra **"Simulación finalizada"**.

SUGERENCIA: utiliza un bucle while donde identificar los estados de los hilos:

```
// Seguimiento de los estados de los hilos
while (moto1.isAlive() || moto2.isAlive() || coche1.isAlive() || coche2.isAlive()) {
   // Seguimiento de los estados de los 4 hilos: muestra por pantalla:
         //Estado de la moto 1: …
         //Estado de la moto 2: …
         //Estado del coche 1: …
         //Estado del coche 2: …
   try {
      Thread.sleep(200); // Pausa para evitar demasiados mensajes
   } catch (InterruptedException e) {
      System.err.println("Seguimiento de estados interrumpido.");
   }
}
```

Entrega del ejercicio 1: Asegúrate de que todo el código esté bien documentado y estructurado. Sube al aula virtual los archivos Moto.java, Coche.java y SimuladorTrafico.java.

Ejercicio 2. Gestión de un sistema de pedidos en una tienda online (5 puntos):

Eres desarrollador de una tienda online y debes implementar un primer sistema para gestionar pedidos. En este sistema, **4 hilos representarán a los clientes que realizan 4 pedidos** y, cuando todos han terminado, otros **4 hilos serán empleados que procesan esos 4 pedidos**. Es una simulación: los empleados no reciben datos de los clientes.

Instrucciones:

a) **(1,5 puntos) Gestión concurrente de clientes con Semaphore (Clientes.java).**
   i. Implementa una clase llamada Clientes que implemente la interfaz Runnable, y reciba el semáforo y su nombre ("Cliente 1"…) en el constructor.
   ii. Cada hilo de esta clase representará un cliente que realiza un pedido.
   iii. Ten en cuenta que utilizarás un objeto Semaphore para posteriormente (en la clase principal) limitar el acceso concurrente a un **máximo de 2 clientes** al mismo tiempo (solo 2 clientes como máximo puedan realizar pedidos simultáneamente).
   iv. Asegúrate de que cada cliente muestre en consola cuándo comienza y termina de realizar un pedido.
   v. Te facilito una estructura para esta clase, donde solo tienes que completar los bloques try, catch, finally.

```
import java.util.concurrent.Semaphore;

public class Clientes implements Runnable {
    private Semaphore semaphore;
    private String nombre;

    public Clientes(Semaphore semaphore, String nombre) {
        this.semaphore = semaphore;
        this.nombre = nombre;
    }

    @Override
    public void run() {
        try {
            // Adquiere un permiso de semáforo

        } catch (InterruptedException e) {
            //Mensaje por consola sobre la interrupción

             return; // no ha cogido el permiso: no hay nada que liberar

        }
        try {
            // Mensaje por consola de que el 'Cliente X está realizando un pedido'

            // Simula el tiempo de realizar el pedido (1 segundo)

            // Mensaje por consola 'Cliente X ha terminado su pedido'

        } catch (InterruptedException e) {
            //Mensaje por consola sobre la interrupción

        } finally {
            // Libera el permiso del semáforo para otros clientes
        }
    }
}
```

b) **(1,75 puntos) Creación y gestión de los empleados (Empleados.java)**.
   i. Implementa una clase llamada Empleados que implemente la interfaz Runnable, y reciba su nombre (“Empleado 1”…) en el constructor.
   ii. Cada hilo de esta clase representará a un empleado que procesa pedidos.
   iii. Simularás que todos los empleados tardan el mismo tiempo de procesamiento de pedidos (1,5 segundos).
   iv. Al terminar, muestra el mensaje: "Empleado X ha terminado de procesar un pedido"

c) **(1,75 puntos) Simulación del sistema de pedidos (TiendaOnline.java).**
   i. Crea una clase principal llamada TiendaOnline que en el main realice lo siguiente:
      - Crea un objeto ‘Semaphore’ **con 2 permisos** (para limitar a 2 clientes realizando el pedido de manera concurrente).
      - **Inicia 4 clientes que generan 4 pedidos**, y utiliza el método join() para esperar a que todos los clientes hayan generado sus pedidos.
      - **Inicia 4 empleados que procesen los 4 pedidos** realizados. **Establece prioridades diferentes para los empleados** (uno de ellos de máxima prioridad, otro de mínima prioridad y dos empleados con prioridad normal).
      - Utiliza el método join() para esperar a que los empleados terminen antes de continuar con la ejecución del programa principal.
      - Muestra un mensaje final: "Todos los pedidos han sido procesados".

Entrega del ejercicio 2: Sube al aula virtual los archivos Clientes.java, Empleados.java y TiendaOnline.java. Asegúrate de que todo el código esté bien documentado y estructurado.